

Examen de la session de rattrapage - Durée (1h30)

Nom et prénom : **N° Apogée**

Exercice 1 (Entourer la/les bonne(s) réponse(s) : plusieurs réponses correctes sont possibles) (5 pts)

1. Que se passe-t-il si fork() échoue ?

- A. Le programme se ferme immédiatement
- B. fork() retourne -1
- C. Un processus fils zombie est créé
- D. errno est mis à 0

2. À quoi sert la fonction execvp() après un fork() ?

- A. Créer un nouveau processus
- B. Terminer le processus père
- C. Remplacer le code du processus courant par un nouveau programme
- D. Suspendre l'exécution d'un processus

3. Quelle est la bonne signature d'un gestionnaire de signal (handler) ?

- A. void handler()
- B. int handler(int sig)
- C. void handler(int sig)
- D. int handler()

4. Si un thread appelle pthread_join() sur lui-même, que se passe-t-il ?

- A. Il attend les autres threads
- B. Il se termine normalement
- C. Il entre en deadlock (blocage)
- D. Il est ignoré

5. Peut-on lire partiellement depuis un pipe ?

- A. Non, il faut tout lire
- B. Oui, avec un buffer plus petit que les données
- C. Seulement si le pipe est nommé
- D. Seulement dans le père

Exercice 2 (Questions de compréhension : donnez une réponse brève et bien précise) (3 pts)

1. Peut-on utiliser un mutex pour synchroniser deux processus différents ? Pourquoi ?

.....
.....

2. Que fait pipe() ?

.....

3. Que se passe-t-il si un signal arrive alors que le processus exécute une section protégée par un mutex ?

.....
.....

Exercice 3 : (5 pts)

Écrire un programme en langage C qui :

- 1. Crée un processus fils à l'aide de fork().
- 2. Le fils simule une tâche longue en affichant toutes les 2 secondes :
 ➤ "Processus fils en cours..."

Il répète cela 10 fois puis termine avec le message :

► "Processus fils terminé."

3. Le père :

- Attend un signal SIGINT (Ctrl+C).
- Lorsqu'il reçoit SIGINT :
 - Il envoie SIGTERM au fils pour lui demander de s'arrêter.
 - Il affiche : "Arrêt demandé. Envoi de SIGTERM au fils."
 - Il attend ensuite la fin du fils.
 - Il affiche : "Processus fils terminé. Fin du programme."
- Si aucun signal n'est reçu, il attend simplement que le fils termine.

Exercice 4 : (7 pts)

On souhaite simuler une situation où plusieurs guichets accèdent concurremment à un compte bancaire unique. Chaque guichet est représenté par un thread, et effectue une série d'opérations (dépôts ou retraits). Les retraits ne sont autorisés que si le solde est suffisant. L'accès au solde doit être protégé pour éviter des erreurs liées aux accès concurrents.

Travail demandé :

Écrire un programme en langage C qui :

1. Initialise un solde partagé à 1000 MAD.
2. Crée 2 threads (guichets).
3. Chaque thread traite une série prédéfinie de 3 opérations (ex. : {+200, -100, -500}).
4. Pour chaque opération :
 - Si c'est un dépôt (valeur positive), le solde est augmenté.
 - Si c'est un retrait (valeur négative), l'opération est acceptée uniquement si le solde reste ≥ 0 .
 - Sinon, elle est refusée.
5. Le solde doit être protégé par un mutex.
6. À la fin, le solde final est affiché par le programme principal.